Memory Layout of C Programs

ECE 2564, Embedded Systems

Fall 2025

Memory Layout

- How are things arranged in memory?
- What goes where?
- It is more efficient for the memory to be partitioned into different segments that have various roles as opposed to dumping all the code and various types of data in one place

Memory Layout

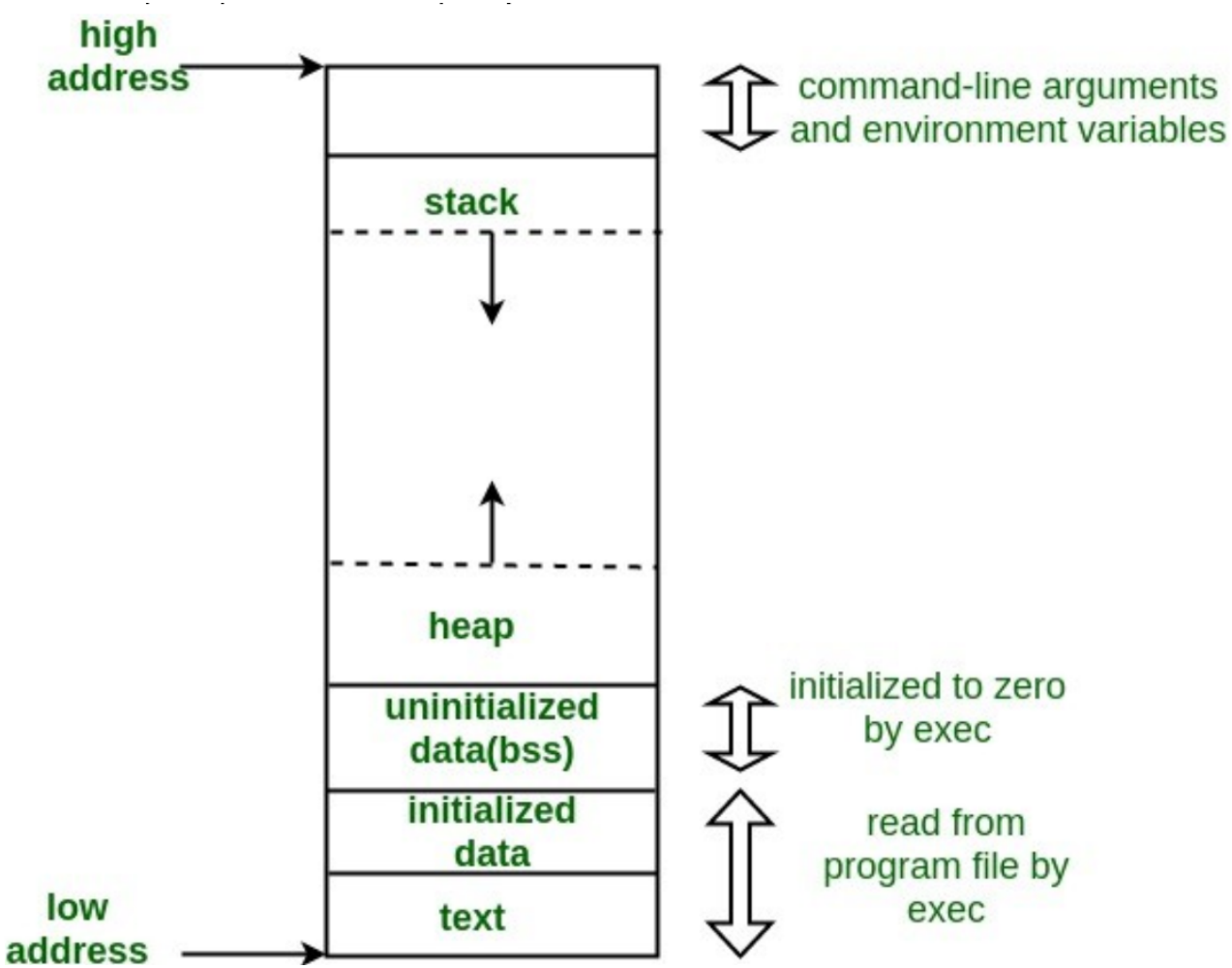
A typical memory representation of a C program consists of the following sections:

- Text segment
- Data segment
- Stack
- Heap

Memory Layout

- Stack: Local variables (non-static), parameters passed to functions, return values from functions, return addresses of functions
  - Function call causes creation of a "stack frame"
  - A function releases the space when it returns
  - Grows from high address downwards
- Heap: Dynamically allocated variables
  - Program does not know the size data structures before run-time
  - Grows from low address upwards
- Data Segment: Global variables and static variables
- Text Segment (also called *code segment*: The program, in the form of machine-language instructions
  - Directly mappable to assembly language

Memory Layout[1]



[1] From https://www.geeksforgeeks.org/memory-layout-of-c-program/

Variables

- When a variable is declared *inside a function*:
  - Normally the variable will reside in the *stack* (**high** address); this local variable is sometimes called an "automatic" variable
  - However, if it has the `static` keyword then it will reside in the "data section" (**low** address)
- Global variables are in the "data section"
- Dynamically allocated data are in the "heap"
  - The code might not know the **size** at compile time

Where is ...?

```
int n = 0;                                          // Where is n?
int main(void) {
    int counter = 0;                                // Where is counter?
    while(n <2) {
        increaseCounter_local();
        increaseCounter_static();
        increaseCounter_pointer(&counter);
        n++;
    }
    char *str = NULL;                               // Where is str?
    str = (char *)malloc(15);                       // Where is *str?
    strcpy(str, "Class example");                   // Where are the 15 characters allocated?
}
void increaseCounter_local() {
    int localCounter = 0;                           // Where is localCounter?
    localCounter++;
}
void increaseCounter_static(){
    static int staticCounter = 0;                   // Where is staticCounter?
    staticCounter++;
}
void increaseCounter_pointer(int *counter_p) {
    *counter_p += 1;                                // Where is counter_p?
}
```

Summary

Memory layout of a typical C program:

- Stack:
  - Local variables (non-static)
  - Parameters passed to functions
  - Return values from functions
  - Return addresses of functions
- Heap: dynamically allocated variables
- Data segment: global variables and static variables
- Text segment (also known as *code segment*): The program in the form of machine-language instructions